DEVIXO SOLUTIONS

AI/ML Internship Program

Task 04

End-to-End ML Application with Deployment

Student Name: Muhammad Ali

Internship Domain: AI/ML

Submission Date: November 1, 2026

Project: Bank Customer Churn Prediction System (classification) on the Kaggle Bank Customer Churn Prediction dataset, with a multi-page Streamlit application

1. Introduction

This report documents Task 04 of the Devixo Solutions AI/ML Internship Program: an end-to-end machine learning application that takes real-world user input, processes it, generates a prediction and presents the result through an interactive interface. It covers data processing, training and comparing five models, hyperparameter tuning, saving the final model with Joblib, a Streamlit application with all five bonus features, deployment preparation and automated testing.

The chosen project is **customer churn prediction**: given a bank customer's details, estimate how likely the customer is to leave the bank. This is a **classification** problem, so accuracy, precision, recall, F1, the confusion matrix and ROC-AUC are used (MAE, MSE, RMSE and R2 from the task sheet apply to regression and are not relevant here).

The next table links each part of the Task 04 sheet to the section of this report that answers it.

Task 04 requirement	Report section
Part 1 - Data processing: load dataset, missing values, duplicates, outliers, encoding, scaling, feature engineering	6, 9, 10, 11
Part 2 - Model development: at least 4 models, comparison, best model, classification metrics	12, 14, 15, 16
Part 3 - Hyperparameter tuning (GridSearchCV / RandomizedSearchCV) and before/after comparison	13
Part 4 - Save the final model (Joblib or Pickle)	17
Part 5 - Streamlit application: inputs, Predict button, prediction, relevant information	18
Part 6 - Deployment	19
Bonus: prediction probability, feature importance, data visualization page, model performance page, multiple model selection	16, 18
Documentation: dataset, workflow, results, limitations, future improvements	3-8, 21-24

2. Objective

The objective is to build a complete, reproducible machine learning application: prepare the data without leaking test information, compare several algorithms fairly, tune them with cross-validation, choose a model by evidence fixed in advance, explain what it learned, check where it fails, save it, and serve it through a professional interface that a non-technical user can operate. The result must be a working application, not only a Jupyter notebook.

3. Dataset Name

Bank Customer Churn Prediction, file `Bank Customer Churn Prediction.csv` (stored in the project as `data/raw/`).

4. Dataset Source

Kaggle dataset 'Bank Customer Churn Prediction'. The dataset description describes it as data for 'ABC Multistate Bank' with customer attributes and a churn flag. The Kaggle page does **not** state how the data was collected, whether it is real or simulated, or under which licence it may be reused. This report therefore does not claim that the customers are real; this is recorded as a limitation (section 22) and should be discussed with the mentor. The licence should be checked on the Kaggle page before the data is redistributed.

5. Dataset Description

Each row is one bank customer. The columns describe the customer (country, gender, age), the relationship with the bank (tenure, number of products, credit card, active membership), financial details (credit score, balance, estimated salary) and the target **churn**, which is 1 when the customer left the bank and 0 otherwise. The description gives no time window for 'left the bank', so predictions are reported as a churn *risk*, not as a prediction for a specific date.

6. Dataset Size

	Raw dataset	After processing
Rows (customers)	10,000	10,000
Columns	12	10 model inputs (customer_id and churn removed), +4 engineered
Exact duplicate rows	0	0
Duplicates ignoring customer_id	0	0
Missing values	0	0 (imputers still in the pipeline)
Train / test rows	-	8,000 / 2,000 (80% / 20%, stratified)
Columns after encoding and scaling	-	21 (training matrix)
File size	about 0.56 MB	-

7. Features/Columns Description

Column	Type	Role / meaning
customer_id	identifier	Unique per row; listed as an unused variable; not a model input
credit_score	integer	Input: customer credit score
country	categorical	Input: France, Germany or Spain
gender	categorical	Input: Female or Male
age	integer	Input: age in years
tenure	integer	Input: years as a customer
balance	float	Input: account balance (many customers have exactly 0)
products_number	integer	Input: number of bank products (1 to 4)
credit_card	binary	Input: 1 if the customer has a credit card
active_member	binary	Input: 1 if the customer is an active member
estimated_salary	float	Input: estimated annual salary
churn	binary	Target: 1 = customer left the bank

8. Methodology

The work was done in Python 3.12.3 with pandas 3.0.2, NumPy 2.4.4, scikit-learn 1.8.0, Matplotlib 3.10.8, Seaborn 0.13.2, Joblib 1.5.3 and Streamlit 1.66.1, in Jupyter notebooks with reusable code in `src/`. The random seed is 42 everywhere. The workflow is:

- **Notebook 01 - Data audit and EDA:** read-only inspection of the raw file; records every data check and the decision it leads to.
- **Notebook 02 - Preparation and features:** clean, split 80/20 **before** learning anything from the data, engineer features, test them with an ablation experiment, and show the preprocessing pipeline.
- **Notebook 03 - Modelling:** baseline models, tuning, before/after comparison, feature importance, selection, error analysis, saving with Joblib and verification by reloading.
- **One pipeline:** feature engineering, imputation, scaling, encoding and the model form one scikit-learn Pipeline fitted on the training data only, so the test set cannot leak and the saved model accepts raw customer inputs.
- **Streamlit app** (`app/`) reads the saved files, and **automated tests** (pytest) check the data, features, saved models and the app.

9. Part 1 - Data Processing

Every action is recorded below with its reason. The raw file was never changed; all work was done on copies.

Step	Action	Result	Reason
1	Loaded the CSV	10,000 rows, 12 columns	Customer data with a binary target
2	Checked for missing values and blank text	0 found	Imputers (median / most frequent) stay inside the pipeline so the saved model is safe if a missing value ever arrives; tested
3	Checked for and removed duplicates	0 found (0 ignoring customer_id)	Done before the split so a repeated customer cannot sit in both train and test
4	Dropped customer_id	1 column	Identifier, not a feature
5	Detected outliers (1.5 x IQR)	credit_score: 15, age: 359	Detection only; values judged plausible and kept (section 10)
6	Stratified split 80/20 (seed 42)	8,000 train / 2,000 test	Done before any statistic is learned; both parts keep the same churn rate
7	Engineered 4 features	balance_zero, balance_to_salary, many_products, age_group	Tested by ablation in section 11
8	Scaled 7 numeric columns (StandardScaler)	fitted on training data only	Needed by Logistic Regression, KNN and SVM; harmless for trees
9	One-hot encoded country, gender, age_group	unseen categories ignored safely	Models need numbers; no false ordering is created
10	Saved the whole pipeline with the model	see section 17	The app applies exactly the same steps as training

The dataset was already clean, so no value needed fixing. The required steps (missing values, duplicates) were still implemented and unit-tested; here they change nothing, and no problems were invented.

10. Exploratory Data Analysis

Target balance. 7,963 customers stayed (79.6%) and 2,037 churned (20.4%). A model that always predicts 'stays' would score 79.7% accuracy on the test set, so accuracy alone is misleading and ROC-AUC, recall and F1 are used as well.

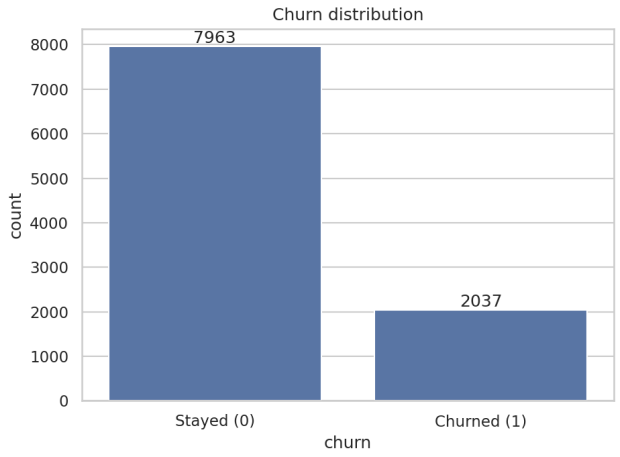


Figure 10.1: churn distribution.

Churn by group. Germany has a churn rate of 32.4% versus 16.2% (France) and 16.7% (Spain). Women churn at 25.1% versus 16.5% for men. Inactive members churn at 26.9% versus 14.3% for active members. Customers with 3 or 4 products churn at 82.7% (n=266) and 100.0% (n=60); these groups are small, so their rates are less reliable. Having a credit card makes almost no difference (20.2% with versus 20.8% without). These are associations in this dataset, not proof of cause.

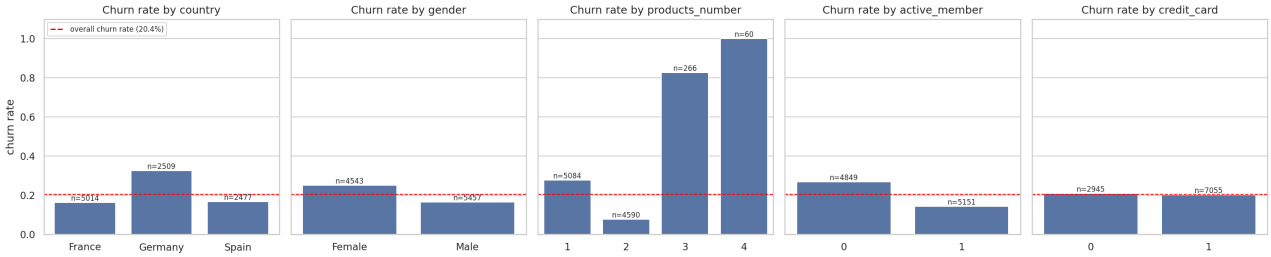


Figure 10.2: churn rate by group (shared y-axis; n = group size; dashed line = overall churn rate).

Age. Churn is 7.5% for customers aged 30 or under, rises to 56.2% in the 51-60 band and falls again to 24.8% above 60. The relationship is not a straight line, which is why age bands were engineered. **Zero balances:** 3,617 customers (36.2%) have a balance of exactly 0. This is a valid group, not missing data; their churn rate is 13.8% versus 24.1% for customers with a balance.

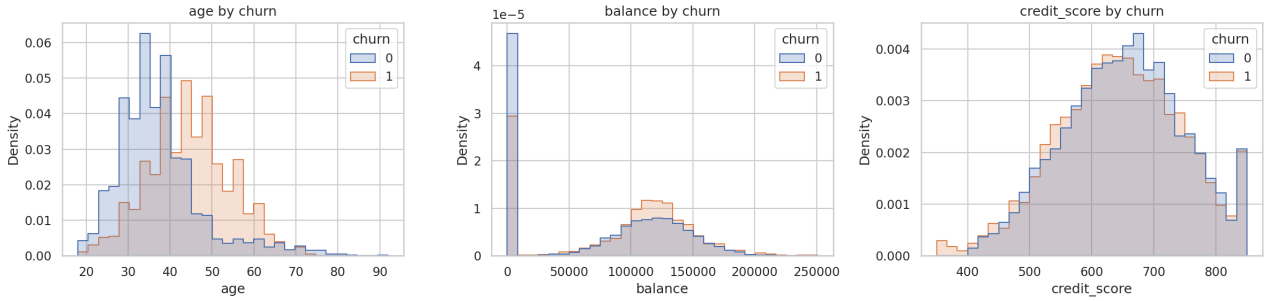


Figure 10.3: age, balance and credit-score distributions by churn.

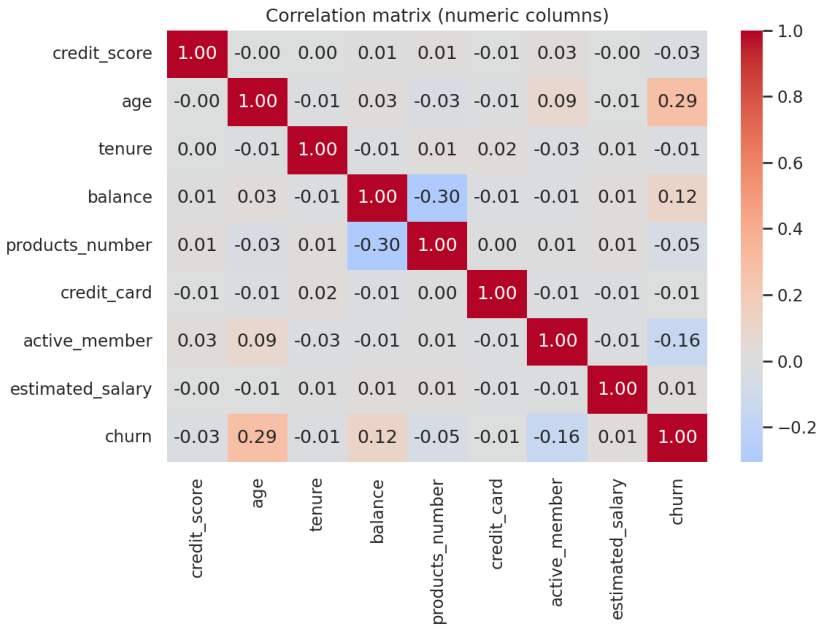


Figure 10.4: correlation matrix (linear association only; not proof of cause).

Outlier detection. The 1.5 x IQR rule flagged values in 2 of 5 numeric columns (credit_score, age):

Column	IQR lower	IQR upper	Outliers	% of rows	Min	Max
credit_score	383	919	15	0.15%	350	850
age	14	62	359	3.59%	18	92
tenure	-3	13	0	0.0%	0	10
balance	-191,466	319,111	0	0.0%	0	250,898

Column	IQR lower	IQR upper	Outliers	% of rows	Min	Max
estimated_salary	-96,577	296,967	0	0.0%	12	199,992

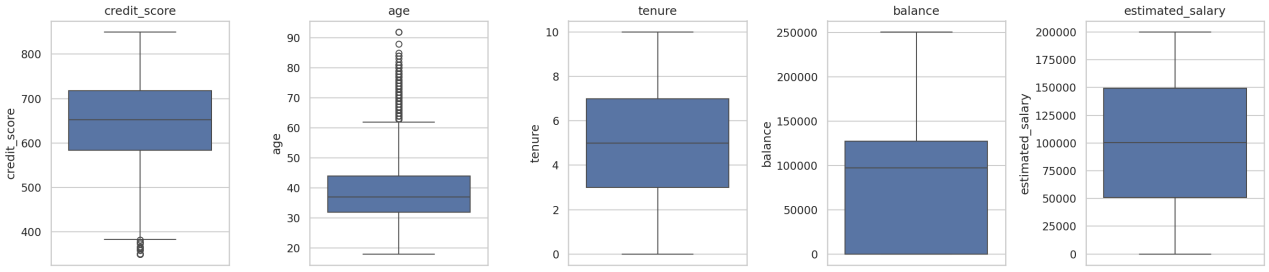


Figure 10.5: boxplots of the numeric columns.

Decision: keep the outliers. The flagged ages go up to 92 and the lowest credit score is 350; these are plausible values for bank customers, not data-entry errors. 151 customers are aged 70 or over and they churn at 9.9% versus 20.4% overall, so deleting them would remove real information. Tree models are not sensitive to such values and the linear and distance-based models use standardised inputs. This was a judgement based on the evidence, and it is documented so it can be reversed.

11. Feature Engineering and Feature Selection

Feature engineering. Four features were created, each from values in the same row only (so nothing can leak from other rows):

Feature	Definition	Why
balance_zero	1 if balance = 0	Large group (36%) with a different churn rate
balance_to_salary	balance / (salary + 1)	Share of income held at the bank
many_products	1 if products_number >= 3	Very different churn behaviour from 1-2 products
age_group	bands: <=30, 31-40, 41-50, 51-60, 60+	Churn peaks in the 50s (not a straight line)

Did they help? (ablation test). Using 5-fold cross-validation on the training data only, Logistic Regression's ROC-AUC rose from 0.763 (no engineered features) to 0.842 (all four); the largest single gain came from many_products (0.830). For Random Forest the change was small (0.847 to 0.850) and within the fold-to-fold variation (about 0.014), as expected because trees can find such splits themselves. balance_zero and balance_to_salary added almost nothing for Logistic Regression on their own. All four were kept because the combined set was best or equal for both models and none hurt performance.

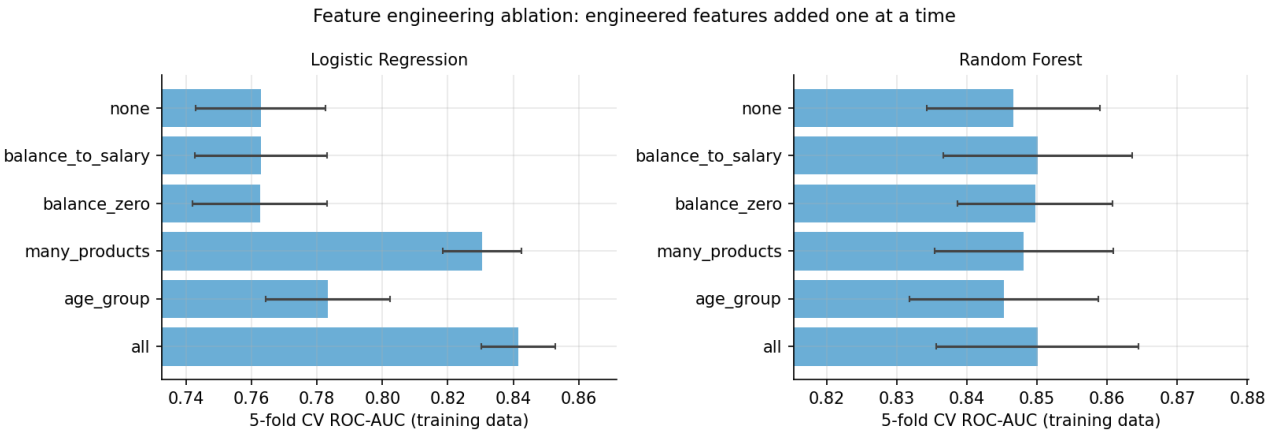


Figure 11.1: cross-validated ROC-AUC with each engineered feature added (training data only).

Feature selection. All ten raw inputs were kept: there are few columns, none is an identifier or a leak, and `customer_id` is the only column removed. Permutation importance in section 16 shows which inputs the final model actually uses.

12. Part 2 - Model Development

Every model is placed in the **same pipeline** and trained on the same training set with the same cross-validation folds, so the comparison is fair. The task requires at least four models; five were trained.

Model	Needs scaling?	Default settings used
Logistic Regression	Yes	max_iter=2000, regularisation C=1
KNN (K-Nearest Neighbours)	Yes	5 neighbours, uniform weights
SVM (RBF kernel)	Yes	C=1, gamma='scale', probability estimates on
Random Forest	No	200 trees
Gradient Boosting	No	scikit-learn defaults

Metrics. Accuracy, precision, recall, F1, the confusion matrix and ROC-AUC are reported (the classification metrics required by the task). ROC-AUC is the selection metric because it does not depend on a threshold and works on imbalanced data. Recall is the share of real churners that are caught; precision is the share of flagged customers that really churn.

Results with default settings (cross-validation on the training set; test metrics at threshold 0.5):

Model	CV ROC-AUC	CV std	Test accuracy	Precision	Recall	F1	Test ROC-AUC
Gradient Boosting	0.863	0.010	0.867	0.780	0.479	0.594	0.870
Random Forest	0.850	0.014	0.864	0.779	0.459	0.578	0.853
KNN	0.785	0.017	0.835	0.638	0.432	0.515	0.788
Logistic Regression	0.842	0.011	0.861	0.789	0.432	0.559	0.849
SVM	0.825	0.017	0.862	0.786	0.442	0.566	0.836

With default settings the weakest model by cross-validated ROC-AUC is KNN (0.785); distance-based and linear models depend on scaled inputs, which the pipeline provides.

13. Part 3 - Hyperparameter Tuning (Before vs After)

Every model was tuned with cross-validation on the **training set only** (5 stratified folds, scoring = ROC-AUC): GridSearchCV for Logistic Regression and KNN (small search spaces, every combination tried) and RandomizedSearchCV for SVM, Random Forest and Gradient Boosting (a fixed number of random combinations). After each search the model is refitted on the whole training set with its best settings. The test set plays no part in tuning.

13.1 Settings searched

Model	Search	Parameters searched
Gradient Boosting	RandomizedSearchCV (20 combinations)	n_estimators {100, 200, 300}; learning_rate {0.03, 0.05, 0.1}; max_depth {2, 3, 4, 5}; subsample {0.7, 0.85, 1.0}; min_samples_leaf {1, 10, 30}
Random Forest	RandomizedSearchCV (20 combinations)	n_estimators {150, 250, 400}; max_depth {None, 6, 8, 10, 12, 16}; min_samples_leaf {1, 2, 4, 8}; max_features {sqrt, 0.5, 0.8}
KNN	GridSearchCV	n_neighbors {5, 9, 15, 21, 31, 41, 61, 81}; weights {uniform, distance}; p {1, 2}
Logistic Regression	GridSearchCV	C {0.01, 0.03, 0.1, 0.3, 1, 3, 10, 30, 100}
SVM	RandomizedSearchCV (8 combinations)	C {0.3, 1, 3, 10, 30}; gamma {scale, 0.01, 0.03, 0.1}

13.2 Results

Model	Best parameters found	CV ROC-AUC	CV std
Gradient Boosting	subsample=0.7, n_estimators=100, min_samples_leaf=30, max_depth=4, learning_rate=0.05	0.8637	0.0083
Random Forest	n_estimators=150, min_samples_leaf=8, max_features=0.5, max_depth=16	0.8613	0.0105
KNN	n_neighbors=61, p=2, weights=distance	0.8422	0.0146
Logistic Regression	C=30	0.8417	0.0109
SVM	gamma=0.01, C=1	0.8327	0.0129

Model	CV AUC before	CV AUC after	Test AUC before	Test AUC after	Change (test AUC)	Test F1 before	Test F1 after
Gradient Boosting	0.8635	0.8637	0.8701	0.8707	+0.0006	0.594	0.590
Random Forest	0.8504	0.8613	0.8531	0.8610	+0.0080	0.578	0.573
KNN	0.7851	0.8422	0.7880	0.8437	+0.0558	0.515	0.472
Logistic Regression	0.8415	0.8417	0.8492	0.8491	-0.0001	0.559	0.561
SVM	0.8255	0.8327	0.8358	0.8463	+0.0105	0.566	0.452

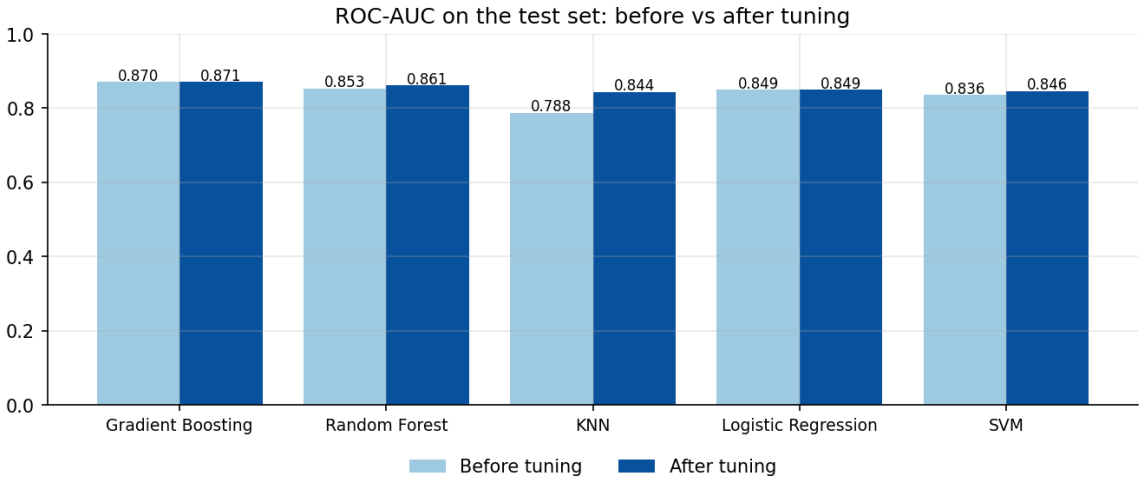


Figure 13.1: test ROC-AUC before and after tuning.

Test ROC-AUC improved noticeably (+0.005 or more) for Random Forest (+0.008), KNN (+0.056), SVM (+0.011). It was essentially unchanged (within 0.005) for Gradient Boosting (+0.001), Logistic Regression (+0.000). F1 at the default 0.5 threshold went *down* after tuning for Random Forest, KNN, SVM. This is expected: tuning optimised ROC-AUC, which ranks customers across all thresholds, while F1 at 0.5 depends on one fixed cut-off; section 14 handles the threshold separately. The 'CV AUC after' value is the best score found during the search and is slightly optimistic by construction; the test columns are the unbiased comparison. The KNN grid was widened to 81 neighbours after a first, narrower grid chose its edge value.

14. Model Selection and Final Test Results

Selection rule, fixed before the test set was used: the model with the highest tuned cross-validated ROC-AUC wins. Gradient Boosting was selected (CV ROC-AUC 0.8637, fold spread 0.0083). Its settings: subsample=0.7, n_estimators=100, min_samples_leaf=30, max_depth=4, learning_rate=0.05. The runner-up, Random Forest, is 0.0024 behind, much less than the fold-to-fold noise, so the top models are effectively close. Changing the choice after seeing test scores would mean selecting a model with the test set, so the rule was kept.

The test set (2,000 customers) was then used once. All tuned models are shown for transparency. The first five metric columns use the standard 0.5 threshold; the last four use each model's own F1-optimised threshold, chosen from out-of-fold predictions on the training data only.

Model	Accuracy	Precision	Recall	F1	ROC-AUC	Threshold	Recall @thr	Precision @thr	F1 @thr
Gradient Boosting (selected)	0.868	0.802	0.467	0.590	0.871	0.29	0.668	0.610	0.638
Random Forest	0.864	0.789	0.450	0.573	0.861	0.30	0.693	0.609	0.648
KNN	0.849	0.818	0.332	0.472	0.844	0.28	0.639	0.551	0.592
Logistic Regression	0.861	0.781	0.437	0.561	0.849	0.29	0.656	0.569	0.610
SVM	0.840	0.746	0.324	0.452	0.846	0.19	0.673	0.567	0.616

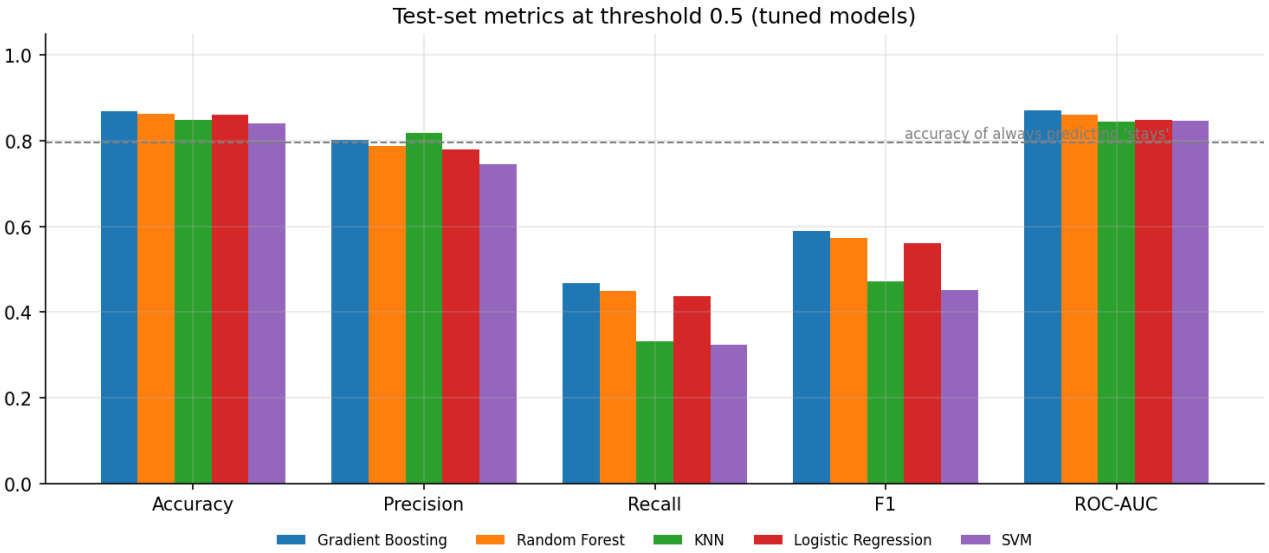


Figure 14.1: test-set metrics of the tuned models (threshold 0.5).

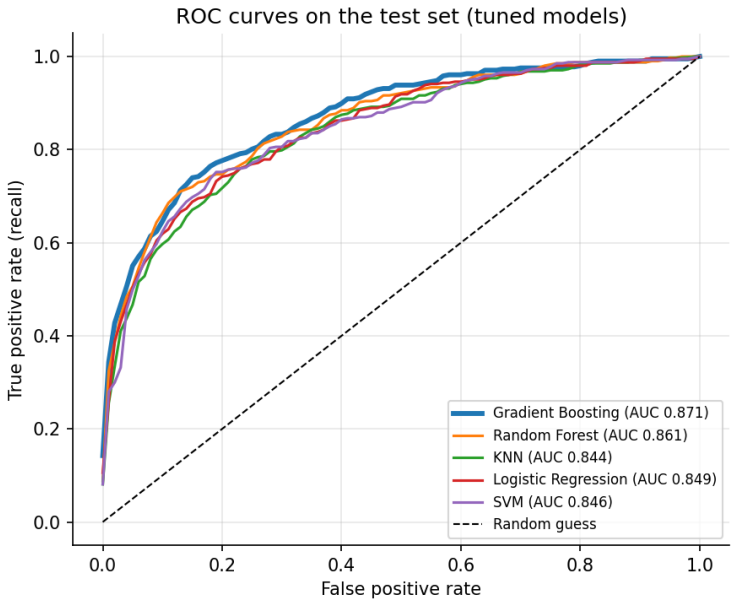


Figure 14.2: ROC curves of the tuned models on the test set.

Honest note. On this particular test set Random Forest reached a slightly higher F1 at its own F1-optimised threshold (0.648 versus 0.638). The gap is small and within the noise of a 2,000-row test set, so it does not change the selection rule fixed in advance; the app lets you switch to Random Forest to compare.

Selected model on the test set. At threshold 0.5 it caught 190 of 407 churners (recall 0.467) and 190 of its 237 churn predictions were correct (precision 0.802); 217 churners were missed. In churn problems a missed churner is often costlier than a false alarm (an assumption: the dataset has no cost information), so a second threshold (0.29) was chosen to maximise F1. There, recall rises to 0.668 with precision 0.610 and F1 0.638. The app labels a customer 'likely to churn' using this second threshold and always shows the raw probability.

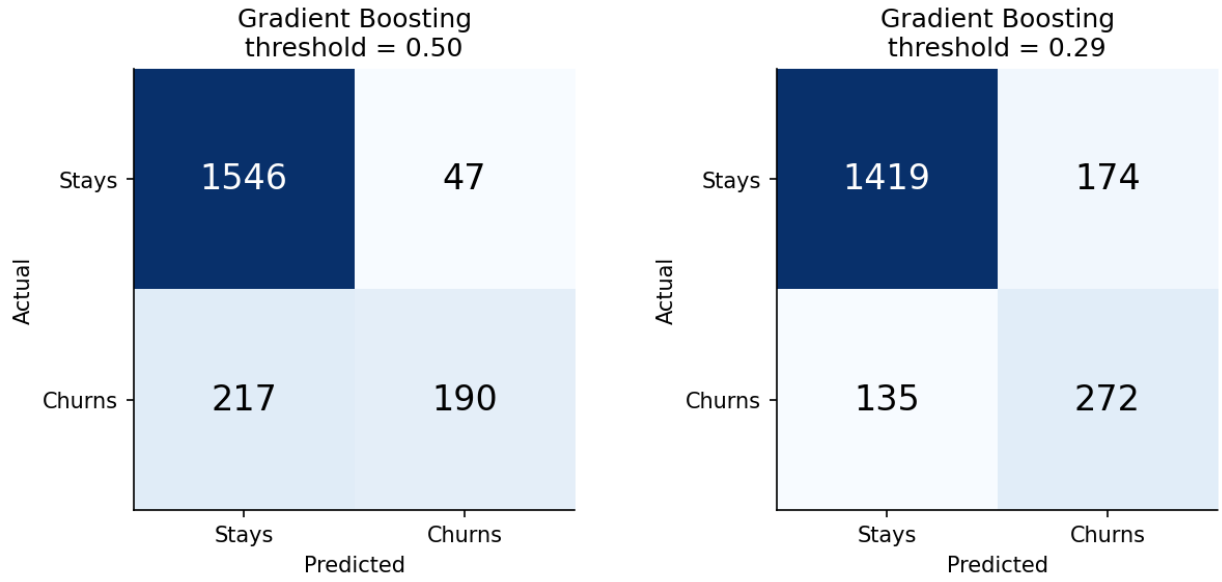


Figure 14.3: confusion matrices of the selected model on the test set (left: threshold 0.5; right: F1-optimised threshold).

15. Error Analysis of the Selected Model

A single score can hide where a model fails, so the selected model (Gradient Boosting, at its F1-optimised threshold 0.29) was examined by customer group and for probability quality on the test set.

Group	Customers	Actual churn rate	Flagged as churn	Recall	Precision
country = France	1021	17.1%	16.8%	0.589	0.599
country = Germany	504	30.0%	38.1%	0.788	0.620
country = Spain	475	17.1%	17.3%	0.617	0.610
gender = Female	930	25.1%	28.2%	0.704	0.626
gender = Male	1070	16.3%	17.2%	0.621	0.587
active_member = 0	968	27.5%	29.9%	0.718	0.661
active_member = 1	1032	13.7%	15.2%	0.574	0.516
products_number = 1	1013	26.7%	33.0%	0.707	0.572
products_number = 2	918	8.4%	4.7%	0.273	0.488
products_number = 3	59	84.7%	100.0%	1.000	0.847
products_number = 4	10	100.0%	100.0%	1.000	1.000
age_group = 31-40	868	11.4%	7.3%	0.323	0.508
age_group = 41-50	492	35.0%	49.4%	0.808	0.572
age_group = 51-60	152	58.6%	67.8%	0.888	0.767
age_group = 60+	89	24.7%	30.3%	0.727	0.593
age_group = <=30	399	6.3%	2.5%	0.240	0.600

Among groups with at least 30 churners in the test set, recall is lowest for **products_number = 2** (0.273, 77 churners) and highest for **products_number = 3** (1.000, 50 churners). Recall differs by group because some groups are easier to separate than others and some are small; the per-group numbers are noisy, so they point to where to look, not to firm conclusions.

Predicted probability bin	Customers	Mean predicted	Observed churn rate
0-10%	952	5.5%	3.9%
10-20%	424	14.2%	13.0%
20-40%	303	28.2%	27.7%
40-60%	140	48.6%	52.9%
60-80%	86	70.5%	77.9%
80-100%	95	88.6%	94.7%

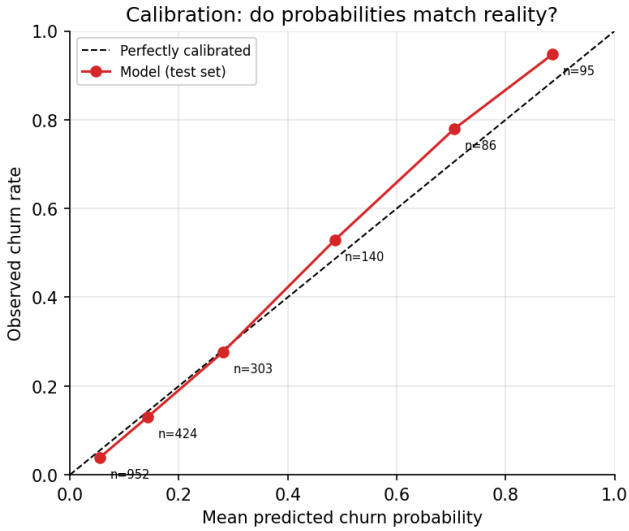


Figure 15.1: calibration of the selected model on the test set.

Are the probabilities trustworthy? In bins with at least 50 customers the mean predicted probability differs from the observed churn rate by at most 7.4 percentage points, so the probabilities are reasonably close to the observed churn rates. The Brier score is 0.099 (lower is better). Bins with few customers are noisy, and this was checked on one test set only.

16. Feature Importance

Permutation importance. One input column of the test set is shuffled at a time and the drop in ROC-AUC is measured (5 repeats). It works for any model, uses the raw input columns and measures predictive value on unseen data. For Gradient Boosting, the most influential inputs were **age** (0.125), **products_number** (0.111), **active_member** (0.030); the least influential were tenure (0.000), credit_card (-0.000).

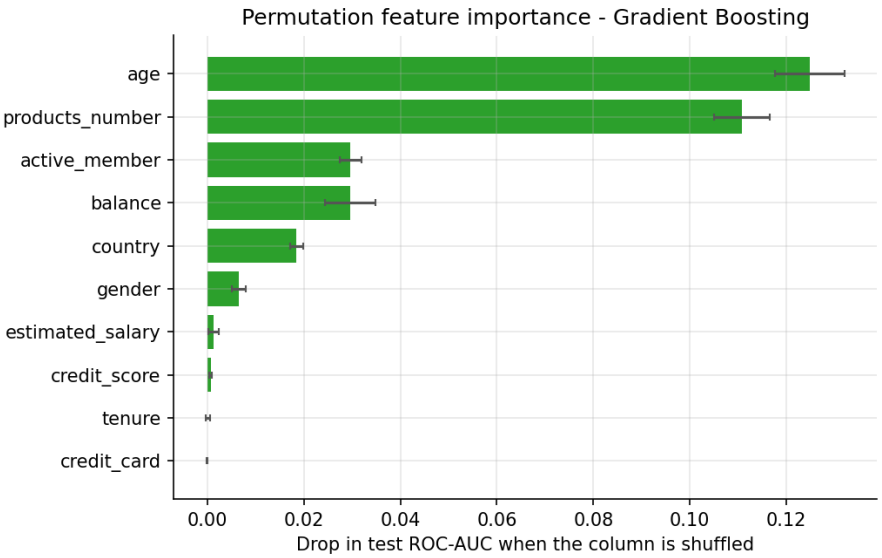


Figure 16.1: permutation importance of the selected model.

Input column	Importance (AUC drop)	Spread (std)
age	0.1249	0.0072
products_number	0.1108	0.0058
active_member	0.0297	0.0023
balance	0.0296	0.0052
country	0.0185	0.0013
gender	0.0065	0.0014
estimated_salary	0.0013	0.0010
credit_score	0.0006	0.0003
tenure	0.0001	0.0004
credit_card	-0.0001	0.0001

Importance shows what the *model* used, not what *causes* customers to leave; correlated inputs can share credit, and values near zero mean the model barely used that column. The app shows this chart per model on its Feature Importance page.

17. Part 4 - Save the Final Model (Joblib)

The **whole pipeline** is saved (feature engineering, imputation, scaling, encoding and the model), so a saved file accepts a raw customer description and returns a probability.

File	Size	Purpose
`models/final_model.joblib`	0.07 MB	The selected model (Joblib, compressed)
`models/all_models/*.joblib`	2.67 MB	All five tuned pipelines (used by the multiple-model selection)
`models/metrics.json`	0.07 MB	Every metric shown in the app and report
`models/reference.json`	0.00 MB	Typical values and segment churn rates from the training split
`models/error_analysis.json`	0.00 MB	Segment and calibration results
`models/feature_ablation.json`	0.00 MB	Feature-engineering experiment

Verification (never trust a save that was not reloaded):

- Notebook 03 reloads every saved model and checks that it predicts the same probabilities as the in-memory model on the whole test set.

- Automated tests check that each model loads, returns valid probabilities, handles missing values, unseen categories and extreme values, and that the metrics stored in metrics.json equal a fresh evaluation of the saved model.
- The models were saved with scikit-learn 1.8.0; requirements.txt pins the library versions because a saved model can only be loaded safely with the same versions. If loading fails on another computer, re-running notebook 03 rebuilds the models with the installed libraries. Joblib/Pickle files must only be loaded from trusted sources.

18. Part 5 - Streamlit Application and Bonus Features

The application (`app/app.py` plus three extra pages) is a working ML application, not only a notebook. It loads the saved pipelines, builds one raw-format customer row from the form and calls the model.

Page	What the user can do
Predict (main page)	Enter 10 customer details, click 'Predict churn risk', and see the verdict, **churn probability** (percentage and progress bar), how it compares with the average customer, an approximate chart of which inputs raise or lower the risk, and the historical churn rates of similar customers. Warns when an input is outside the range seen in training.
Data Visualization (bonus)	Filter by country, gender and age; churn overview, churn rate by group, numeric distributions, correlation.
Model Performance (bonus)	Comparison table, before/after tuning, confusion matrices, ROC curves, calibration and the feature-engineering ablation, all read from saved files.
Feature Importance (bonus)	Permutation importance for the selected model and a comparison across all five models.
Sidebar on every page (bonus)	**Multiple model selection:** any of the five tuned models can be chosen; the best is the default.

How the 'what is driving this prediction' chart works. For each input, only that value is replaced with the typical value from the training data and the change in predicted probability is shown. It is a simple model-based explanation that ignores interactions and is not evidence of cause. **Usability details:** the form starts with a typical customer; results appear only after the button is clicked; tables with highlighted cells set both background and text colour so they are readable in light and dark themes.

Run it with `streamlit run app/app.py`. The app is covered by automated tests (section 20) and was started on a real Streamlit server to check that it loads. No screenshot is included in this report.

19. Part 6 - Deployment

The application is deployed at: `https://bank-churn-predictor-gb.streamlit.app`

- Push the project to a public GitHub repository (the models/ folder must be included).
- Open share.streamlit.io, sign in with GitHub and choose New app.
- Select the repository and branch and set the main file path to app/app.py; choose Python 3.12 in the advanced settings.
- Open the live URL, test a prediction on every page, and add the URL to the README and to this report.

20. Quality Assurance and Testing

An automated test suite (pytest) checks the data, the pipeline, the saved models, the notebooks and the app:

- **Data:** schema, no missing values or duplicates, cleaning logic (duplicates and missing targets are removed), the split is stratified, reproducible and has no overlap, customer_id is never a model input.
- **Features:** correct values, age-band boundaries (18, 30, 31, 40, 41, 50, 51, 60, 61, 100), no NaN or infinity, input data not modified, zero salary does not divide by zero.
- **Models:** all five load; probabilities are valid; minimum, maximum, risky-combination, unseen-category and missing-value inputs do not crash; stored metrics equal a fresh evaluation; confusion-matrix totals equal the

test-set size; the stored best model has the highest CV score.

- **Notebooks:** all three notebooks exist, are executed and contain no error output.
- **App (Streamlit AppTest):** every page loads without errors; the form and Predict button exist; no result appears before the click; a click gives exactly one verdict with a probability and progress bar; every one of the five models can be selected; extreme inputs give a warning, not a crash; a high-risk profile scores higher than a low-risk profile.

Test run output:

```
pytest results (run on 2026-11-01):
tests/test_data_and_features.py: 20 passed in 3.19s
tests/test_models.py: 46 passed in 3.02s
tests/test_notebooks.py: 7 passed in 1.28s
tests/test_app.py: 14 passed in 11.97s
ALL TESTS: 87 passed in 12.16s
```

Automated tests cannot replace looking at the running app in a browser; screenshots should be captured and added to the README.

21. Conclusion and Recommendation

A complete, leak-free pipeline was built on 10,000 customers: the data was audited, split before anything was learned from it, features were engineered and tested, five models were compared and tuned with cross-validation, one was selected by a rule fixed in advance, its errors and probabilities were examined, and everything was saved and put behind a Streamlit application.

Recommendation: use the tuned Gradient Boosting. On the untouched test set it reaches ROC-AUC 0.871; at its F1-optimised threshold it catches 66.8% of churners with 61.0% precision. It should be used to *rank* customers by churn risk and prioritise retention effort, not as a certain prediction for any one customer.

Lessons from this project: the split must come before any fitting; one pipeline prevents leakage and makes saving simple; accuracy misleads on imbalanced data; features should be kept or dropped on evidence; tuning does not always help; models can be close to tied, and the honest answer is to say so; and a saved model must be reloaded and tested.

22. Limitations

- **Data provenance:** the source and licence are not stated on Kaggle, and it is not confirmed that the data is real; the task asks for a real-world dataset.
- **Churn definition:** no time window is given, so the app reports a risk, not a prediction for a date.
- **Small groups:** customers with 3-4 products are few (266 and 60 customers), so patterns there are less reliable.
- **Single split:** a 2,000-row test set gives noisy estimates; small differences between models should not be over-interpreted.
- **Calibration** was checked on one test set only; bins with few customers are noisy.
- **Fairness:** gender and country are model inputs because they are in the dataset; in a real bank their use would need legal and ethical review.
- **Explanations** (importance, driving factors) describe model behaviour, not causes.
- **Random split, not a time split:** the data has no dates, so performance on future customers is untested.
- **Modest tuning:** 8-20 random combinations for the larger search spaces; a longer search might change small differences between models.
- **Versions:** models were trained with scikit-learn 1.8.0 and only load reliably with that version (pinned).

23. Future Improvements

- Confirm the data source and licence, and test on genuinely independent data.
- Calibrate the probabilities (for example isotonic regression) and choose the threshold from real retention costs.
- Use repeated cross-validation or a larger hold-out for more stable model comparison; try a wider or Bayesian hyperparameter search.
- Add fairness checks across country and gender groups.
- Deploy the app, monitor incoming inputs against the training ranges, and retrain when the data drifts.
- Add per-customer explanations with a dedicated method such as SHAP, and optional models such as XGBoost.

24. Major Findings

- The dataset has 10,000 customers with no missing values or duplicates; 20% churn, so accuracy is misleading (always predicting 'stays' scores 79.7%).
- Churn is higher for Germany, women, inactive members and customers with 3-4 products; churn peaks in the 51-60 age band.
- many_products clearly helped Logistic Regression in the ablation test (0.763 to 0.830 ROC-AUC); for Random Forest the engineered features made little difference.
- Tuning helped some models noticeably and others not at all (see section 13); Gradient Boosting was selected by cross-validation.
- Test result of the selected model: ROC-AUC 0.871, recall 0.467 and precision 0.802 at 0.5; recall 0.668 and precision 0.610 at the F1-optimised threshold.
- The most influential inputs were age, products_number, active_member.
- The saved Joblib pipelines reload to identical predictions, and the Streamlit app and all tests pass.

25. Requirements Checklist and Status

Requirement	Status	Evidence
Load a real-world dataset	PARTIAL	Kaggle bank churn data (10,000 rows); real vs simulated origin not stated by the source (sections 4, 22).
Handle missing values	PASS	Checked (0 found); imputers in the pipeline; tested with a missing input.
Remove duplicates	PASS	Checked (0 found); clean() removes them; unit-tested.
Detect outliers	PASS	IQR table and boxplots (section 10); decision to keep documented.
Encode categorical variables	PASS	One-hot encoding for country, gender, age_group.
Scale numerical features	PASS	StandardScaler inside the pipeline.
Feature engineering	PASS	Four features with ablation evidence (section 11).
Train at least 4 models, compare, select best	PASS	5 models; best = Gradient Boosting by cross-validated ROC-AUC (sections 12, 14).
Classification metrics (Accuracy, Precision, Recall, F1, Confusion Matrix, ROC-AUC)	PASS	Sections 12 and 14; all in metrics.json.
Hyperparameter tuning (Grid or Randomized)	PASS	Both used (section 13).
Compare before/after tuning	PASS	Section 13 tables and figure.
Save final model (Joblib or Pickle)	PASS	models/final_model.joblib plus all_models/*.joblib (full pipelines), reload-verified.
Streamlit app: inputs, button, prediction, relevant information	PASS	app/app.py; verified by automated app tests.
Bonus: prediction probability	PASS	Percentage and progress bar.
Bonus: feature importance	PASS	Feature Importance page.
Bonus: data visualization page	PASS	app/pages/1_Data_Visualization.py.
Bonus: model performance page	PASS	app/pages/2_Model_Performance.py.
Bonus: multiple model selection	PASS	Sidebar selector on every page.
Training code in Jupyter notebooks	PASS	notebooks/01-03 (.ipynb), executed with saved outputs.
Deploy online if possible	DEPLOYED	Live at https://bank-churn-predictor-gb.streamlit.app.

26. Appendix - Key Code

The code below is copied from the project files at build time (text, not screenshots). The notebooks in the repository contain all cells with their saved outputs.

A.1 Feature engineering (src/features.py)

```
"""Feature engineering.

`add_features` is used as the first step of the sklearn Pipeline. It only uses
values from the same row (no statistics learned from other rows), so it cannot
leak information from the test set.
"""
import numpy as np
import pandas as pd

AGE_BINS = [0, 30, 40, 50, 60, 200]
AGE_LABELS = ["<=30", "31-40", "41-50", "51-60", "60+"]

# Names of the engineered columns, grouped by how the preprocessor treats them.
ENGINEERED_NUMERIC = ["balance_to_salary"]
ENGINEERED_BINARY = ["balance_zero", "many_products"]
ENGINEERED_CATEGORICAL = ["age_group"]
ALL_ENGINEERED = ENGINEERED_NUMERIC + ENGINEERED_BINARY + ENGINEERED_CATEGORICAL


def add_features(X: pd.DataFrame) -> pd.DataFrame:
    """Return a copy of X with four extra columns.

    balance_zero      1 if the account balance is exactly 0. Zero-balance
                      customers are a large group (~36%) with a different churn
                      rate, and 'balance == 0' is a different signal from
                      'balance is small'.
    balance_to_salary balance divided by salary: how much of a customer's income
                      is held at this bank (+1 avoids any division by zero).
    many_products     1 if the customer has 3 or more products. These groups are
                      small but behave very differently from 1-2 products.
    age_group         age in bands (<=30, 31-40, 41-50, 51-60, 60+). Churn does
                      not rise steadily with age (it peaks in the 50s), so bands
                      let simple models such as Logistic Regression capture that.
    """
    X = X.copy()
    X["balance_zero"] = (X["balance"] == 0).astype(int)
    X["balance_to_salary"] = X["balance"] / (X["estimated_salary"] + 1)
    X["many_products"] = (X["products_number"] >= 3).astype(int)
    X["age_group"] = pd.cut(X["age"], bins=AGE_BINS, labels=AGE_LABELS).astype(str)
    X["balance_to_salary"] = X["balance_to_salary"].replace([np.inf, -np.inf], np.nan)
    return X
```

Listing A.1: feature engineering used inside the pipeline

A.2 Preprocessing and model pipeline (src/pipeline.py)

```
"""Builds the full sklearn Pipeline: feature engineering -> preprocessing -> model.

The saved model is this whole Pipeline, so the Streamlit app applies exactly the
same transformations as training. Imputers, scalers and encoders are fitted only
on the training data when `.fit()` is called on the training split.
"""
from sklearn.compose import ColumnTransformer
from sklearn.ensemble import GradientBoostingClassifier, RandomForestClassifier
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import FunctionTransformer, OneHotEncoder, StandardScaler
from sklearn.svm import SVC

from src.data_utils import RANDOM_STATE
from src.features import (ALL_ENGINEERED, ENGINEERED_BINARY, ENGINEERED_CATEGORICAL,
                          ENGINEERED_NUMERIC, add_features)

MODEL_NAMES = ["Logistic Regression", "KNN", "SVM", "Random Forest", "Gradient Boosting"]

BASE_NUMERIC = ["credit_score", "age", "tenure", "balance", "estimated_salary", "products_number"]
BASE_CATEGORICAL = ["country", "gender"]
BASE_BINARY = ["credit_card", "active_member"]


def build_preprocessor(engineered=None) -> ColumnTransformer:
    """Impute + scale numeric columns, impute + one-hot encode categoricals.

    `engineered` chooses which engineered columns to include (default: all).
    The imputers do nothing on this dataset (it has no missing values) but make
    the saved model safe if a missing value ever arrives.
    """
    engineered = list(ALL_ENGINEERED if engineered is None else engineered)
    numeric = BASE_NUMERIC + [c for c in engineered if c in ENGINEERED_NUMERIC]
    categorical = BASE_CATEGORICAL + [c for c in engineered if c in ENGINEERED_CATEGORICAL]
    binary = BASE_BINARY + [c for c in engineered if c in ENGINEERED_BINARY]

    return ColumnTransformer([
        ("num", Pipeline([("impute", SimpleImputer(strategy="median")),
                          ("scale", StandardScaler())]), numeric),
        ("cat", Pipeline([("impute", SimpleImputer(strategy="most_frequent")),
                          ("onehot", OneHotEncoder(handle_unknown="ignore"))]), categorical),
        ("bin", SimpleImputer(strategy="most_frequent"), binary),
    ])


def make_model(name: str, **params):
    """Create an untrained model with sensible defaults (params override them)."""
    if name == "Logistic Regression":
        defaults = dict(max_iter=2000, random_state=RANDOM_STATE)
        return LogisticRegression(**{**defaults, **params})
    if name == "KNN":
        return KNeighborsClassifier(**params)
    if name == "SVM":
        defaults = dict(probability=True, cache_size=1000, random_state=RANDOM_STATE)
        return SVC(**{**defaults, **params})
    if name == "Random Forest":
        defaults = dict(n_estimators=200, random_state=RANDOM_STATE, n_jobs=-1)
        return RandomForestClassifier(**{**defaults, **params})
    if name == "Gradient Boosting":
        defaults = dict(random_state=RANDOM_STATE)
        return GradientBoostingClassifier(**{**defaults, **params})
    raise ValueError(f"Unknown model: {name}")


def build_pipeline(name: str, engineered=None, **model_params) -> Pipeline:
    return Pipeline([
        ("features", FunctionTransformer(add_features)),
        ("prep", build_preprocessor(engineered)),
        ("model", make_model(name, **model_params)),
    ])
```

Listing A.2: the common pipeline used for every model

A.3 Hyperparameter tuning (notebook 03)

```
SEARCH_SPACES = {
    "Logistic Regression": ("grid", {"model__C": [0.01, 0.03, 0.1, 0.3, 1, 3, 10, 30, 100]}, None),
    "KNN": ("grid", {"model__n_neighbors": [5, 9, 15, 21, 31, 41, 61, 81],
                     "model__weights": ["uniform", "distance"],
                     "model__p": [1, 2]}, None),
    "SVM": ("random", {"model__C": [0.3, 1, 3, 10, 30],
                       "model__gamma": ["scale", 0.01, 0.03, 0.1]}, 8),
    "Random Forest": ("random", {"model__n_estimators": [150, 250, 400],
                                 "model__max_depth": [None, 6, 8, 10, 12, 16],
                                 "model__min_samples_leaf": [1, 2, 4, 8],
                                 "model__max_features": ["sqrt", 0.5, 0.8]}, 20),
    "Gradient Boosting": ("random", {"model__n_estimators": [100, 200, 300],
                                     "model__learning_rate": [0.03, 0.05, 0.1],
                                     "model__max_depth": [2, 3, 4, 5],
                                     "model__subsample": [0.7, 0.85, 1.0],
                                     "model__min_samples_leaf": [1, 10, 30]}, 20),
}

tuned = {}          # evaluation results per model
tuned_models = {}   # fitted pipelines per model

for name in MODEL_NAMES:
    t0 = time.time()
    kind, space, n_iter = SEARCH_SPACES[name]

    search_params = {"probability": False} if name == "SVM" else {}
    search_pipe = build_pipeline(name, **search_params)
    if kind == "grid":
        search = GridSearchCV(search_pipe, space, scoring="roc_auc", cv=CV, n_jobs=1)
    else:
        search = RandomizedSearchCV(search_pipe, space, n_iter=n_iter, scoring="roc_auc",
                                    cv=CV, n_jobs=1, random_state=RANDOM_STATE)
    search.fit(X_train, y_train)

    best_params = {k.replace("model__", ""): (v.item() if hasattr(v, "item") else v)
                   for k, v in search.best_params_.items()}

    # Rebuild the final model with the best settings (probability=True for the SVM) and fit on ALL training
        rows
    final = build_pipeline(name, **best_params).fit(X_train, y_train)
    res = evaluate_pipeline(final, X_train, y_train, X_test, y_test, CV)
    res.update({
        "search_type": "GridSearchCV" if kind == "grid" else "RandomizedSearchCV",
        "n_candidates": int(len(search.cv_results_["params"])),
        "best_params": best_params,
        "cv_roc_auc_mean": float(search.best_score_),
        "cv_roc_auc_std": float(search.cv_results_["std_test_score"][search.best_index_]),
    })
    tuned[name] = res
    tuned_models[name] = final
    print(f"{name:20s} {res['search_type']:20s} best CV AUC = {search.best_score_:.4f} | "
          f"test AUC = {res['test_default_threshold']['roc_auc']:.4f} ({time.time()-t0:.0f}s)")
    print("   best parameters:", best_params)
```

Listing A.3: search spaces and the tuning loop

A.4 Saving the models (notebook 03)

```
for name in MODEL_NAMES:
    joblib.dump(tuned_models[name], ALL_MODELS_DIR / f"{slug(name)}.joblib", compress=3)
joblib.dump(tuned_models[best_name], MODELS_DIR / "final_model.joblib", compress=3)

results = {n: {"baseline": baseline[n], "tuned": tuned[n]} for n in MODEL_NAMES}
metrics = {
    "meta": {
        "trained_at_utc": datetime.now(timezone.utc).isoformat(timespec="seconds"),
        "quick_run": False,
        "python": platform.python_version(), "sklearn": sklearn.__version__,
        "pandas": pd.__version__, "numpy": np.__version__,
        "random_state": RANDOM_STATE,
        "n_rows_raw": int(len(raw)), "n_rows_clean": int(len(df)),
        "n_train": int(len(X_train)), "n_test": int(len(X_test)),
        "churn_rate_train": float(y_train.mean()), "churn_rate_test": float(y_test.mean()),
        "majority_class_accuracy_test": float(1 - y_test.mean()),
        "cv_folds": 5, "primary_metric": "roc_auc",
        "data_quality": quality, "outliers_iqr": outliers,
        "training_seconds": round(time.time() - T_START),
    },
    "model_order": ranking,
    "best_model": best_name,
    "selection_reason": reason,
    "models": results,
}
(MODELS_DIR / "metrics.json").write_text(json.dumps(metrics, indent=2))
(MODELS_DIR / "reference.json").write_text(json.dumps(reference_profile(X_train, y_train), indent=2))
(MODELS_DIR / "error_analysis.json").write_text(json.dumps({"model": best_name, **ea}, indent=2))

for p in sorted(MODELS_DIR.rglob("*")):
    if p.is_file():
        print(f"{str(p.relative_to(ROOT)):45s} {p.stat().st_size/1e6:7.2f} MB")
```

Listing A.4: saving the pipelines, metrics and reference files

A.5 Prediction explanation used by the app (app/app_utils.py)

```
def factor_analysis(model, inputs: dict, ref: dict) -> pd.DataFrame:
    """Approximate 'what pushes this customer's risk up or down'.

    For each input we swap ONLY that value for the typical training value and see
    how the predicted probability changes. A positive change means the customer's
    own value raises the risk compared with a typical customer. This is a
    model-based explanation (not a causal one) and ignores interactions.
    """
    typical = reference_inputs(ref)
    p_orig = predict(model, inputs)
    rows = []
    for col in RAW_FEATURES:
        if inputs[col] == typical[col]:
            continue
        tmp = dict(inputs)
        tmp[col] = typical[col]
        rows.append({"feature": col, "your_value": inputs[col], "typical_value": typical[col],
                     "effect_on_probability": p_orig - predict(model, tmp)})
    out = pd.DataFrame(rows)
    if out.empty:
        return out
    return out.reindex(out["effect_on_probability"].abs().sort_values(ascending=False).index)
```

Listing A.5: approximate 'what drives this prediction' analysis

27. GitHub Repository Link

https://github.com/Al7bh/Devixo-Internship (folder: Task_04_ML_App)

The task folder contains the three executed notebooks, src/ (reusable code), the data, the saved models in models/, the figures in reports/, the tests, the Streamlit app in app/, this report, requirements.txt and the README.